

SMART PLACEMENT TRACKER

A SERVERLESS AWS ARCHITECTURE FOR CAMPUS PLACEMENT PREPARATION AND TRACKING

Prepared in the partial fulfillment of the Summer Internship Program on AWS



AT

NAIPUNYAM

Under the guidance of

Mrs. Bethala Sumana , APSSDC

Mr. Juttuka Lovababu , APSSDC

submitted by

Vankayala Jyothi, 24A81A43C7, sri Vasavi engineering college
Tadepalligudem (Batch-1)

Yedida Devi Uma Mani Alekhya, 24A81A43D1, sri Vasavi engineering
college Tadepalligudem (Batch - 1)

Malleswarapu Bhuvana Sri Vijaya, 24A81A4398, Sri Vasavi engineering
college Tadepalligudem (Batch - 2)

Madasi Priya Dharsini, 24A81A4396, Sri Vasavi engineering college
Tadepalligudem (Batch - 2)

ABSTRACT

This project presents the implementation of Smart Placement Tracker, a serverless, full-stack web platform built on AWS to help engineering students prepare for campus placements. The primary goal is to give students a single dashboard to manage their profile, track company applications, log aptitude and coding practice, and receive an automatically calculated Placement Readiness Score, while relying entirely on managed, event-driven AWS services rather than provisioned servers.

The system's frontend is a responsive HTML, CSS, and JavaScript single-site application hosted on Amazon S3 as a static website. All backend logic runs on AWS Lambda functions written in Python, invoked through Amazon API Gateway. Student data — profiles, applications, aptitude scores, and coding practice logs — is stored in Amazon DynamoDB, while resumes are uploaded directly to Amazon S3 using presigned URLs generated by a dedicated Lambda function, keeping file transfer off the compute layer entirely.

Authentication is implemented using salted, hashed passwords and signed bearer tokens issued by a login Lambda function, avoiding the overhead of running a dedicated auth server. A readiness-scoring module aggregates a student's coding activity, aptitude performance, resume status, and profile completeness into a single weighted score, and an analytics module aggregates this data further into charts covering application trends, selection rate, and skill growth over time.

This architecture is fully serverless, using Lambda for computation, API Gateway for request routing, DynamoDB for real-time data access, S3 for both static hosting and file storage, and IAM for secure, least-privilege permissions between services. The solution offers high scalability, low operational overhead, and near-zero idle cost, making it well suited to a single-user or small-cohort student tool. The project demonstrates how a serverless AWS architecture can be applied to a real, everyday student problem, while leaving a clear path for future enhancements such as an AI-powered career assistant, automated deadline notifications, and an admin analytics dashboard.

TABLE OF CONTENTS

1. INTRODUCTION.....	4-5
2. METHODOLOGY.....	6-8
3. TECHNOLOGY STACK.....	9-10
4. SYSTEM DESIGN / ARCHITECTURE.....	11-15
5. IMPLEMENTATION.....	16-17
6. RESULTS.....	18-22
7. CONCLUSION.....	23

1. INTRODUCTION

In the digital era, where agility, automation, and real-time decision-making have become essential to student success, effective placement preparation has emerged as a critical need for engineering students. This project, titled Smart Placement Tracker, harnesses the power of serverless computing on Amazon Web Services (AWS) to deliver a robust, scalable, and fully automated platform for tracking and improving a student's placement readiness. It stands as a testament to how cloud technologies can transform a fragmented, spreadsheet-driven process into an intelligent, single-dashboard experience with minimal infrastructure overhead.

Engineering students preparing for campus placements typically juggle several disconnected tools: spreadsheets for tracking company applications, separate coding-practice platforms, aptitude test scores scattered across notebooks, and resumes that are never quite up to date. This fragmentation makes it hard for a student to answer a simple question honestly: "How placement-ready am I, right now?" Smart Placement Tracker was built to answer exactly that question, in one place, updated automatically as the student logs their progress.

Designed with simplicity, automation, and clarity at its core, the system allows a student to register, build out a profile, log company applications with their current status, record aptitude and coding practice sessions, and upload a resume — all of which feed into a single, auto-calculated Placement Readiness Score presented as a GO / CAUTION / HOLD indicator, similar to a pre-flight checklist, rather than a generic progress bar.

What sets this project apart is its strategic use of AWS's serverless services, eliminating the need for server provisioning, manual scaling, or ongoing infrastructure maintenance. Each Lambda function is invoked only when a student interacts with the app, enabling optimal resource usage and near-zero idle cost. Additionally, the use of AWS IAM guarantees secure, least-privilege interactions between services, while Amazon CloudWatch facilitates monitoring and debugging of every function.

This report provides a comprehensive overview of the system's design, architecture, and implementation, highlighting the decision-making process, technical integrations, and benefits realized through a serverless approach. The architecture promotes modularity and extensibility, enabling easy integration with future enhancements such as an AI career-guidance chatbot, automated deadline notifications via Amazon SNS, and an admin dashboard for placement cell staff.

2. METHODOLOGY

The development of Smart Placement Tracker followed a structured and iterative methodology to ensure the successful fulfillment of project objectives. The methodology was divided into key phases, including requirements gathering, system design, implementation, testing, deployment, and refinement, aligned with best practices in cloud-native, serverless development.

2.1. Requirements Gathering

The project began with a review of the everyday challenges students face while preparing for placements: scattered application tracking, no single view of readiness, and no automated way to flag weak areas. This helped define the core objectives: let a student manage their profile and resume in one place, track every company application through its full lifecycle, log aptitude and coding practice, and see an automatically computed readiness score with actionable next steps.

2.2. Design

Based on the identified requirements, a cloud-native, event-driven architecture was designed using AWS services. The workflow was defined to include Amazon S3 for static frontend hosting and resume storage, Amazon API Gateway as the single entry point for all requests, AWS Lambda for all business logic, and Amazon DynamoDB for data storage. IAM roles were planned to ensure secure, least-privilege interactions between services, and particular attention was given to keeping every table keyed by student ID so that queries never require an expensive full-table scan.

2.3. Implementation

The implementation phase focused on translating the design into a fully functional serverless application. Registration and login were implemented as dedicated Lambda functions issuing signed bearer tokens; profile, company-tracker, aptitude, and coding-tracker Lambda functions were implemented with consistent request validation and DynamoDB access patterns; and a dedicated Lambda function was built to hand out short-lived, presigned S3 URLs so that resume uploads go directly from the browser to S3 without passing through Lambda at all.

2.4. Database Management

A NoSQL approach was adopted for efficient and scalable data storage using Amazon DynamoDB, with five tables: Users, StudentProfile, Applications, AptitudeProgress, and CodingProgress. Each application-data table uses the student's user ID as its partition key, ensuring that every read is a cheap, targeted query rather than a scan. Data validation is performed inside each Lambda function before any write, ensuring integrity without needing a schema-enforcing database engine.

2.5. Testing

Multiple levels of testing were carried out to validate functionality, error handling, and system performance. Each Lambda function was tested independently for input validation and correct DynamoDB access. Integration testing confirmed that the frontend, API Gateway, Lambda, DynamoDB, and S3 all worked together end-to-end, covering registration, login, profile updates, company tracking, aptitude and coding logging, resume upload, and the

aggregated dashboard and analytics views.

2.6. Deployment

The system was deployed on AWS using the console, with DynamoDB tables, IAM roles, Lambda functions, and API Gateway routes configured manually and documented step-by-step for repeatability. Environment variables were used throughout so that table names, the resume bucket, and the signing secret could be changed without touching code. Amazon CloudWatch was used to monitor logs and diagnose issues such as integration misconfiguration during rollout.

2.7. Iterative Refinement

Throughout the development cycle, iterative feedback loops were used to refine the system — correcting API Gateway route-to-Lambda mappings, adding a shared Lambda layer so common utility code did not need to be duplicated across every function, and enabling CORS on the resume S3 bucket once direct browser uploads were introduced. The architecture was reviewed periodically to ensure adherence to serverless best practices and remained ready for future enhancement.

3. TECHNOLOGY STACK

Smart Placement Tracker was developed using a modern, cloud-native technology stack that supports event-driven architecture, real-time data access, and low-maintenance scalability.

3.1. Cloud Platform

- Amazon Web Services (AWS): the entire backend is hosted on AWS, leveraging its serverless services for hosting, computation, database management, and file storage.

3.2. AWS Services

- Amazon S3: hosts the static frontend as a website, and stores uploaded resumes via presigned URLs.
- Amazon API Gateway (HTTP API): routes every frontend request to the correct Lambda function and manages CORS.
- AWS Lambda: executes all backend logic — authentication, profile management, company tracking, aptitude and coding logging, resume URL generation, dashboard aggregation, and analytics.
- Amazon DynamoDB: NoSQL database storing users, profiles, applications, aptitude records, and coding records, keyed by student ID for fast, scalable access.
- AWS IAM: provides secure access control between services by defining least-privilege roles and policies.
- Amazon CloudWatch: monitors Lambda execution, captures logs, and supports debugging and optimization.

3.3. Programming & Frontend

- Python 3.12 (Lambda runtime): used for all serverless backend logic.
- Boto3 SDK: the Python SDK for AWS, used within Lambda to interact with DynamoDB and S3.
- HTML5, CSS3, vanilla JavaScript: a dependency-free frontend, deployable directly to S3 with no build step.

- Chart.js: renders trend, distribution, and comparison charts across the aptitude, coding, and analytics pages.

3.4. Security & Access

IAM roles and policies were configured so that each Lambda function has only the permissions it needs — for example, the resume-upload function can only write to the resume bucket, and no function has broader DynamoDB access than the single table it needs to read or write.

3.5. Monitoring & Logging

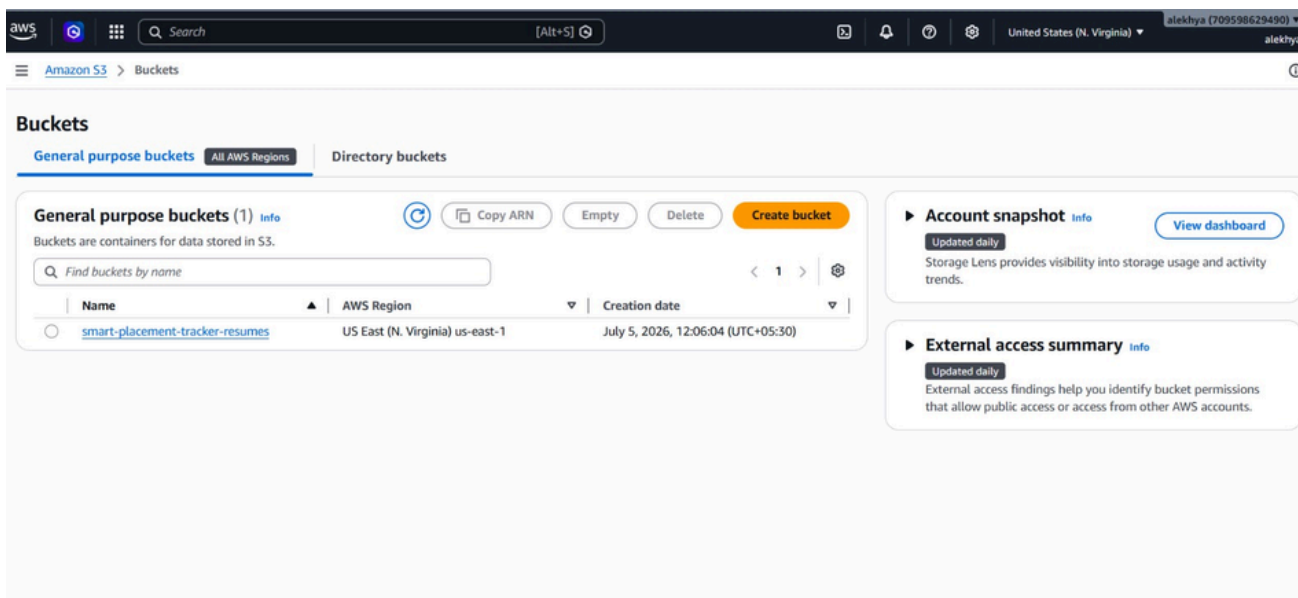
- CloudWatch Logs: captures execution logs from every Lambda invocation for tracking and troubleshooting.
- CloudWatch Metrics: provides real-time data on function invocations, errors, and duration.

4. SYSTEM DESIGN / ARCHITECTURE

Smart Placement Tracker is designed as an event-driven, cloud-native solution that automates placement-readiness tracking end to end. The architecture adopts a serverless model, ensuring scalability, minimal infrastructure maintenance, and cost-efficiency. The system follows a multi-tier design comprising the presentation layer, API layer, processing logic, data storage, and file storage, all powered by managed AWS services.

4.1. Presentation Layer – Amazon S3 Static Website

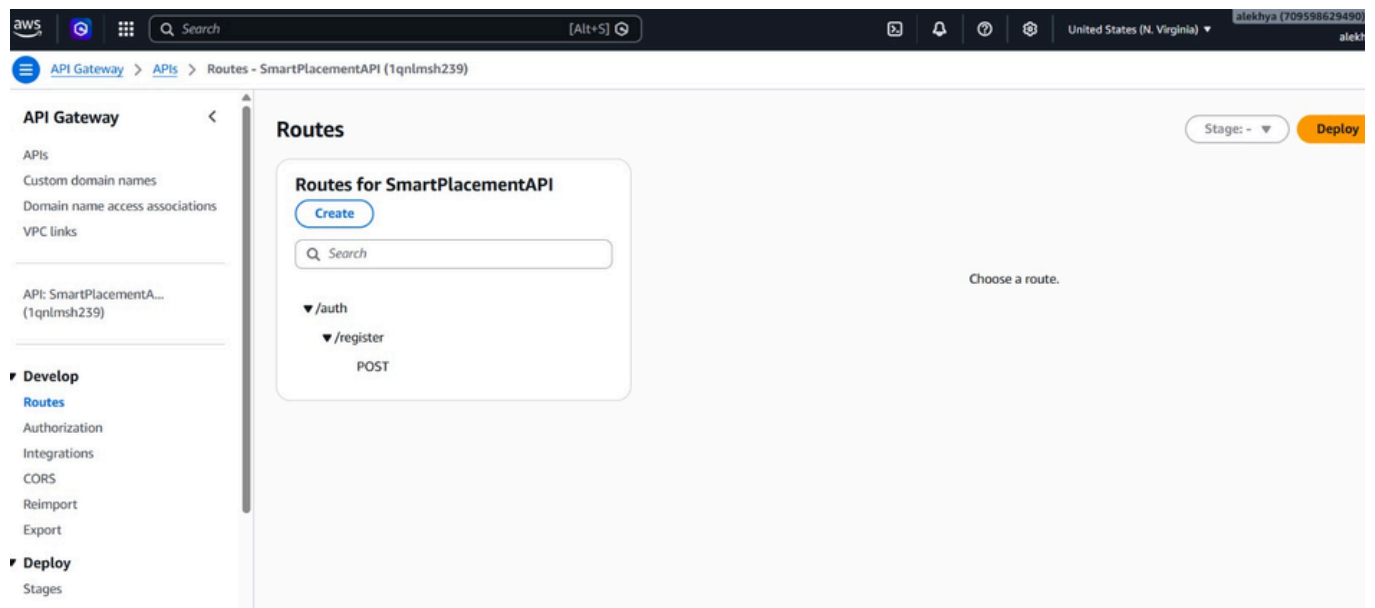
The frontend – login/registration, dashboard, profile, company tracker, aptitude tracker, coding tracker, and analytics pages – is a static HTML/CSS/JavaScript site hosted directly on an Amazon S3 bucket configured for static website hosting. There is no build step and no server to manage; deployment is simply uploading the frontend folder to the bucket.



S3 static website hosting configuration for the frontend bucket

4.2. API Layer — Amazon API Gateway

An HTTP API in Amazon API Gateway exposes every backend route — `/auth/login`, `/auth/register`, `/profile`, `/companies`, `/aptitude`, `/coding`, `/resume/upload-url`, `/dashboard`, and `/analytics` — and integrates each one with its corresponding Lambda function. CORS is configured at the API Gateway level so the S3-hosted frontend can call it from the browser.



API Gateway routes for SmartPlacementAPI

4.3. Processing Logic — AWS Lambda

Each route is backed by a dedicated Python Lambda function performing a focused task: validating and hashing passwords during registration, issuing signed tokens during login, reading and writing student profile data, running full CRUD on tracked company applications, appending aptitude and coding practice entries, generating presigned S3 upload URLs for resumes, and aggregating a student's data into the dashboard summary and analytics views. Shared logic — DynamoDB access, response formatting, and token verification — is packaged as a single Lambda layer so it is not duplicated across functions.

Functions (15) Last fetched 7/6/2026, 8:31:10 PM

Functions table allows you to query function details. Use Ctrl + K to start a in-console resource search.

Search by attributes or search by keyword

☐	Function name	Description	Package type	Runtime	Type	Last modified
☐	spt-aptitude-list	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-companies-delete	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-dashboard-get	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-auth-register	-	Zip	Python 3.12	Standard	22 hours ago
☐	spt-companies-update	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-analytics-get	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-coding-create	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-coding-list	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-profile-update	-	Zip	Python 3.12	Standard	1 day ago
☐	spt-profile-get	-	Zip	Python 3.12	Standard	1 day ago

Choose a tutorial
Learn how to implement common use cases in AWS Lambda.

Create a simple web app ^

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

[Start tutorial](#)

Lambda functions list for the Smart Placement Tracker backend

4.4. Data Storage — Amazon DynamoDB

Student data is stored across five DynamoDB tables — Users, StudentProfile, Applications, AptitudeProgress, and CodingProgress — each keyed by the student's user ID so that every read is a fast, targeted query rather than a full-table scan.

DynamoDB > **Tables**

Tables (5) Info Last updated July 5, 2026, 12:01 (UTC+5:30)

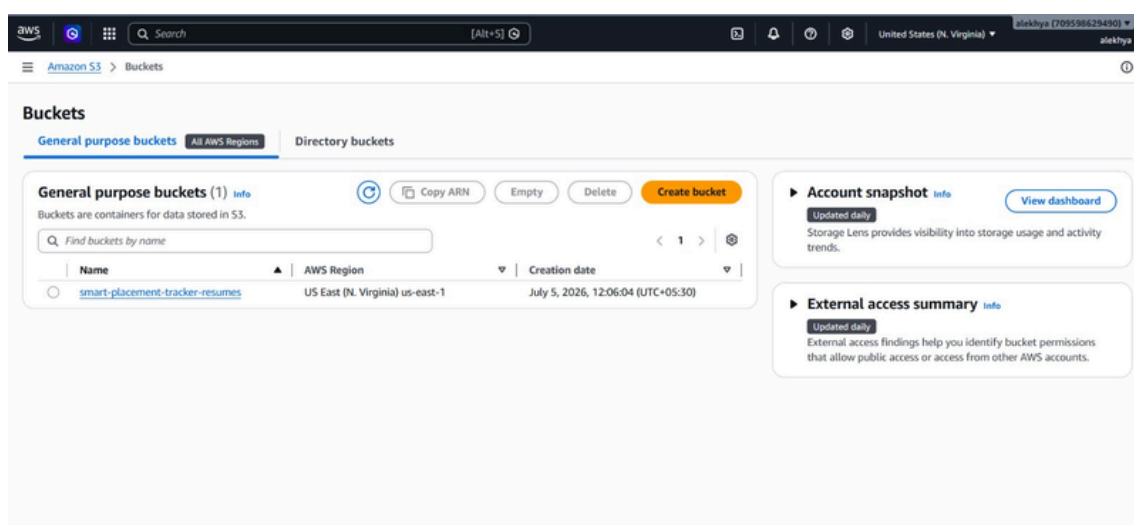
Find tables Filter by tag Any tag key Filter by tag value Any tag value

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite
☐	SPT_Applications	Active	userid (\$)	applicationid (\$)	0	0	Off	☆
☐	SPT_AptitudeProgress	Active	userid (\$)	entryid (\$)	0	0	Off	☆
☐	SPT_CodingProgress	Active	userid (\$)	entryid (\$)	0	0	Off	☆
☐	SPT_StudentProfile	Active	userid (\$)	-	0	0	Off	☆
☐	SPT_Users	Active	email (\$)	-	0	0	Off	☆

DynamoDB tables for Smart Placement Tracker

4.5. File Storage — Amazon S3 (Resumes)

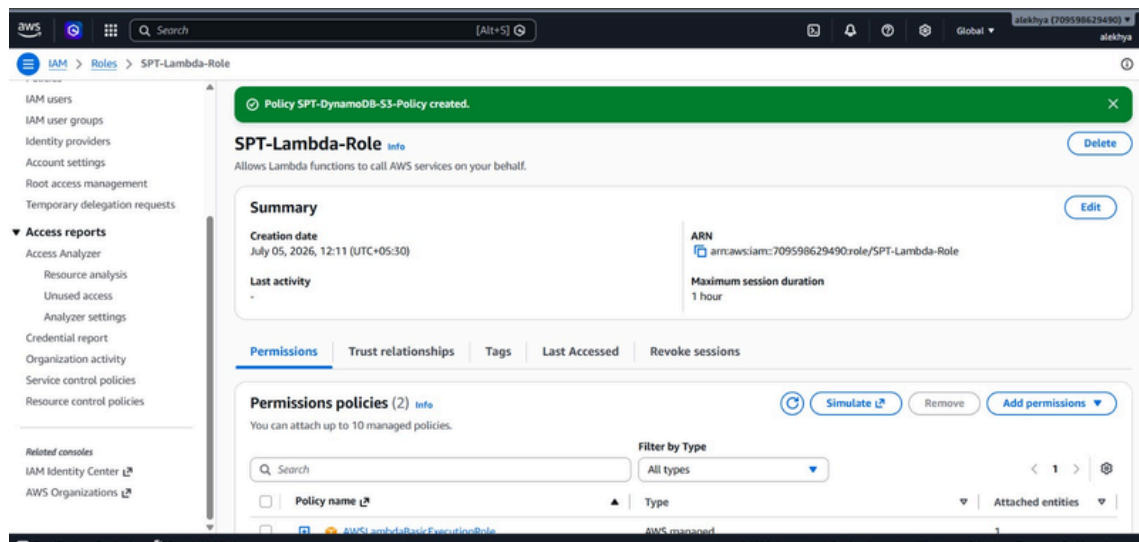
Resumes are DynamoDB tables for Smart Placement Tracker uploaded directly from the browser to a dedicated S3 bucket using a presigned PUT URL generated by a Lambda function, so the file itself never passes through Lambda or API Gateway. CORS is configured on the bucket to permit PUT requests from the frontend's origin.



S3 bucket CORS configuration for resume uploads

4.6. Security and Access Control — AWS IAM

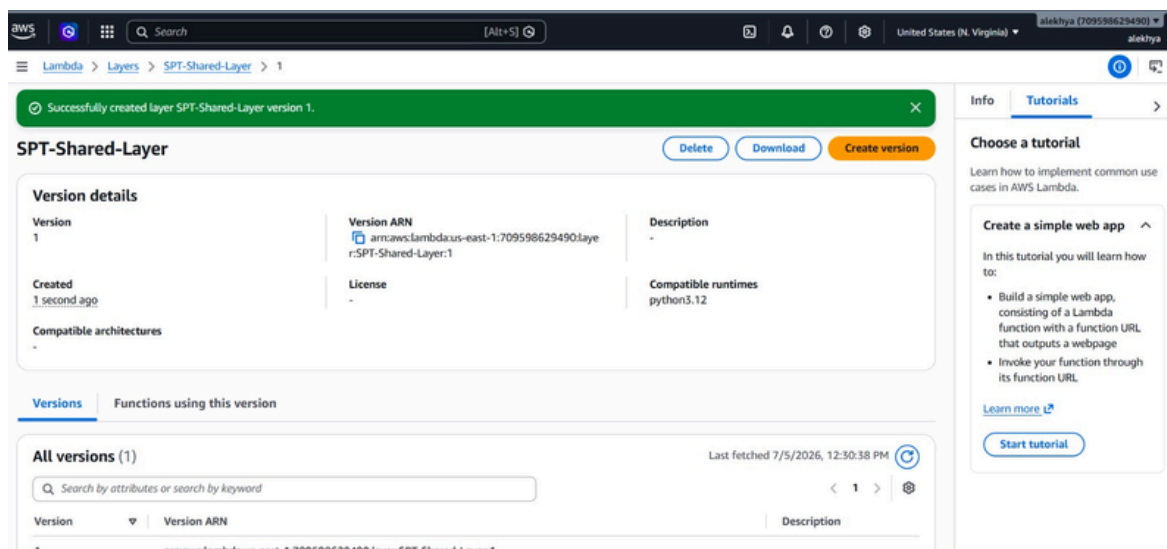
AWS IAM is used to securely grant permissions between services: each Lambda function's execution role is scoped to only the DynamoDB tables or S3 bucket it needs, authentication uses salted password hashes and signed bearer tokens, and least-privilege access policies are enforced throughout to reduce the system's attack surface.



IAM roles and permission policies for the Lambda execution roles

4.7. Monitoring and Logging – Amazon CloudWatch

Amazon CloudWatch is integrated for real-time monitoring of every Lambda function's execution, viewing logs for debugging and error tracing, and setting alerts for function failures or unusual activity.



CloudWatch log group for a Smart Placement Tracker Lambda function

5. IMPLEMENTATION

The implementation of Smart Placement Tracker translates the architectural design into a fully functional cloud-based platform. This section outlines how each module was configured and developed to work together in an automated, event-driven workflow.

5.1. Authentication

Registration validates all required fields, hashes the password using PBKDF2 with a per-user salt, stores the user record in the Users table, seeds a blank StudentProfile row, and returns a signed bearer token. Login verifies the submitted password against the stored hash and returns a fresh signed token. All subsequent requests carry this token in the Authorization header and are verified by every protected Lambda function before it touches DynamoDB.

5.2. Student Profile

The profile module lets a student maintain their name, contact details, college, branch, CGPA, skills, programming languages, certifications, projects, and links to LinkedIn, GitHub, LeetCode, and HackerRank. A resume can be uploaded through a presigned S3 URL, and the resulting file URL is written back onto the profile record.

5.3. Company Application Tracker

Students can add every company they apply to, with role, package, location, application date, deadline, and a status drawn from a fixed pipeline — Interested, Applied, OA Scheduled, OA Cleared, Interview Scheduled, HR Round, Selected, or Rejected. The tracker supports search, status filtering, and a running summary of total, selected, pending, and rejected applications.

5.4. Aptitude Tracker

Aptitude scores are logged by category — Quantitative, Logical Reasoning, or Verbal — and the tracker computes weekly and monthly activity, average score, and best score, visualized as a trend chart.

5.5. Coding Tracker

Coding practice sessions are logged by platform (LeetCode, GeeksforGeeks, HackerRank, CodeChef, or Codeforces), problem count, and difficulty. The tracker computes a daily streak, a 90-day practice heatmap, and a difficulty-distribution chart

5.6. Placement Readiness Score

A dedicated scoring module combines coding activity, aptitude performance, resume status, profile completeness, projects, and certifications into a single weighted score out of 100, displayed on the dashboard as a semi-circular gauge with GO, CAUTION, and HOLD zones, along with a personalized list of next actions to raise the score.

5.7. Analytics

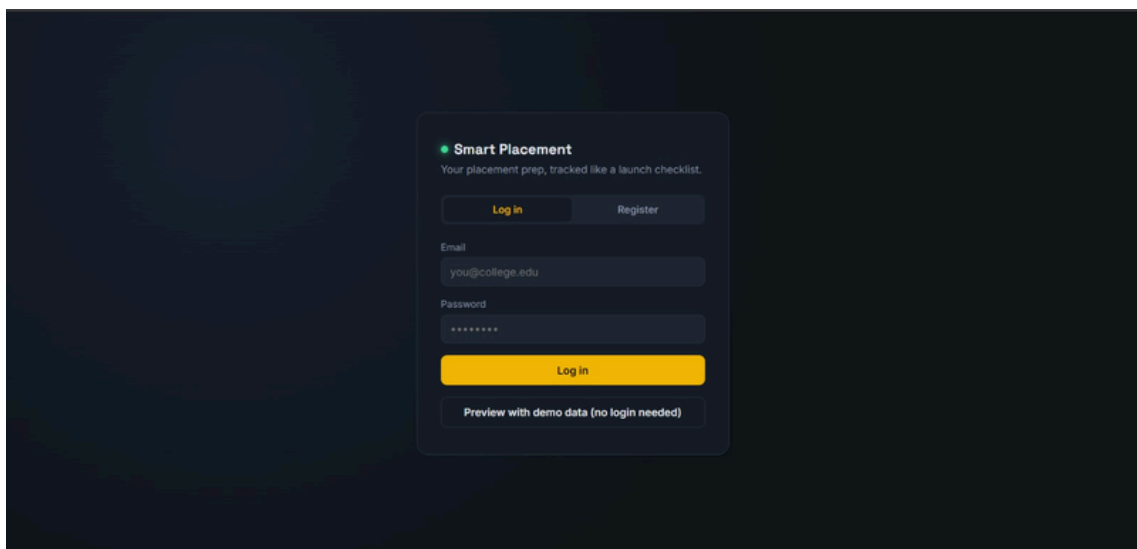
The analytics page aggregates a student's data into charts covering applications per month, selection rate, coding growth by week, and aptitude improvement by category, giving a longer-term view than the day-to-day tracker pages.

6. RESULTS

The implementation of Smart Placement Tracker yielded several successful outcomes that align with the project's objectives. The system was tested under multiple scenarios to validate functionality, performance, and reliability of the serverless architecture.

6.1. Working Authentication Flow

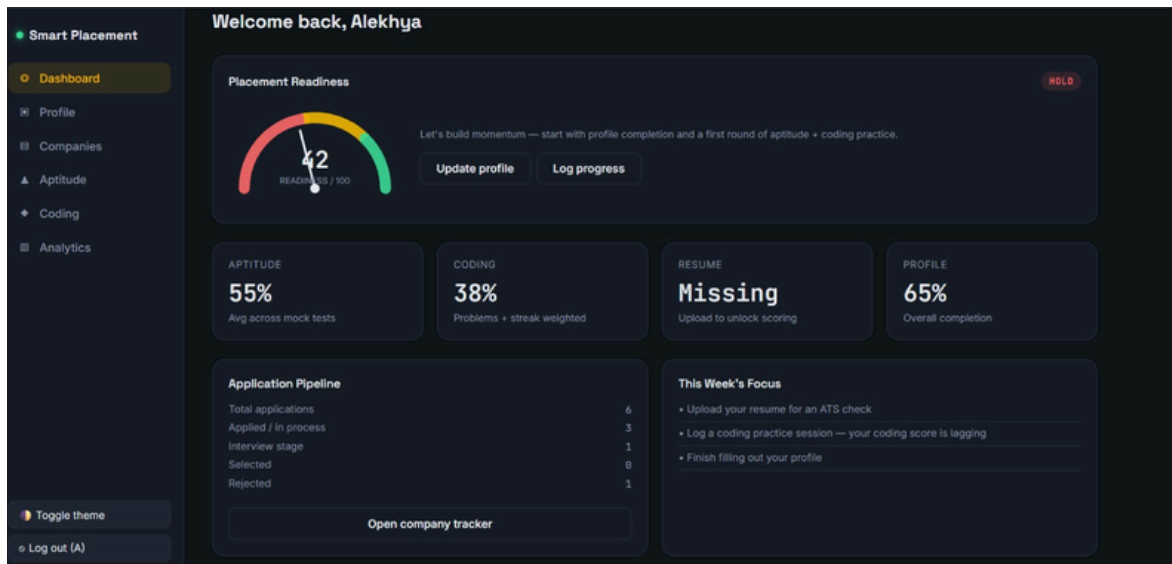
Registration and login were tested end-to-end: a new account can be created, a signed token is issued and stored client-side, and the token correctly authorizes subsequent profile, company, aptitude, and coding requests.



Login / Register screen

6.2. Dashboard and Readiness Score

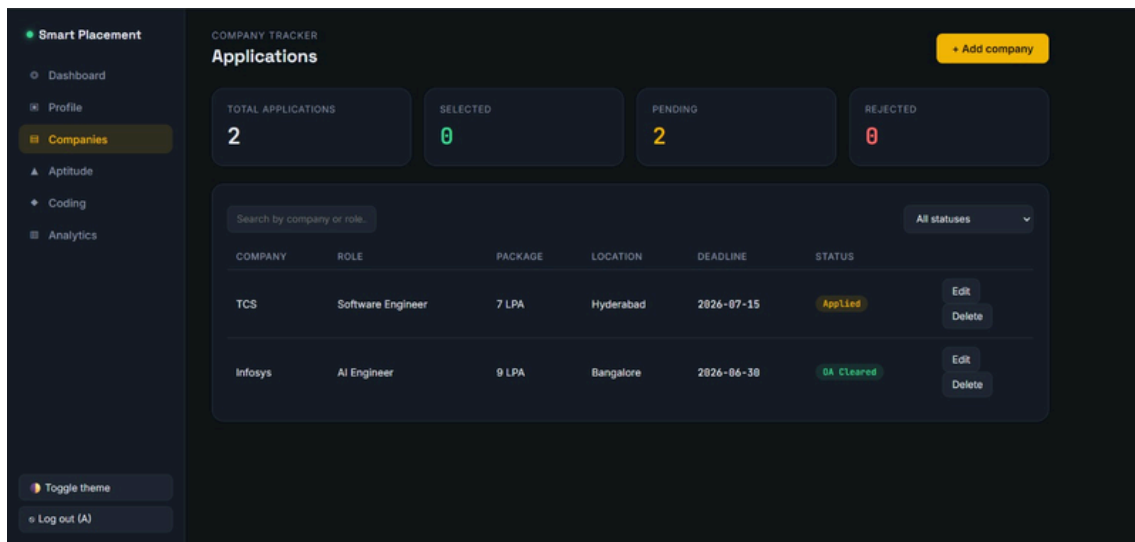
The dashboard correctly aggregates a student's profile, aptitude, coding, and application data into a single readiness score, displayed with the GO / CAUTION / HOLD gauge and a focused list of next actions.



Dashboard page

6.3. Company Application Tracking

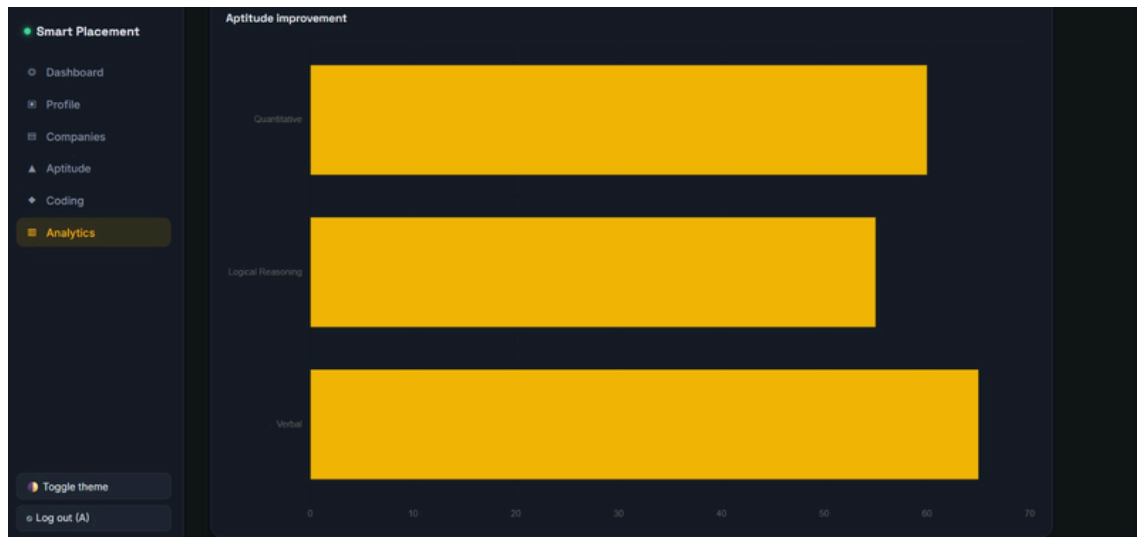
Applications can be added, edited, searched, and filtered by status, with the summary cards updating immediately to reflect total, selected, pending, and rejected counts.



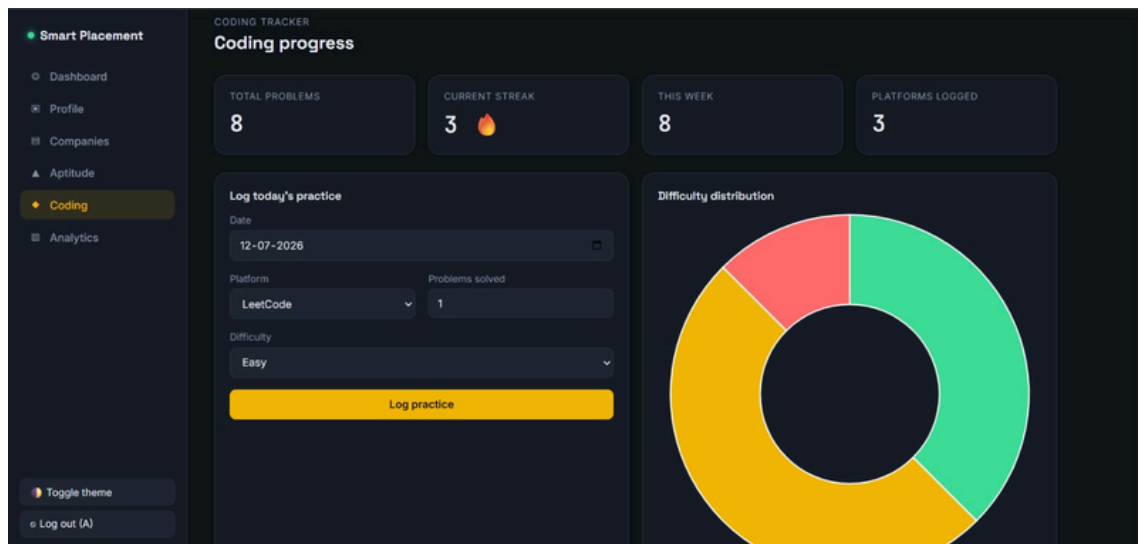
Company tracker page with some applications

6.4. Aptitude and Coding Tracking

Aptitude and coding entries are logged and immediately reflected in their respective trend chart, streak counter, and heatmap, confirming that the DynamoDB write path and the chart-rendering path are both working correctly.



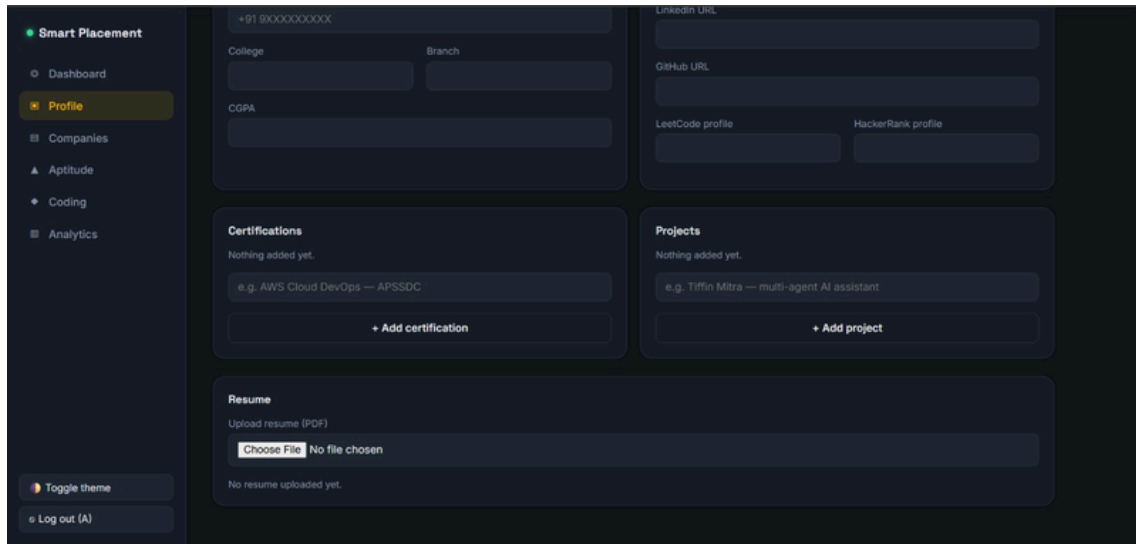
Aptitude tracker trend chart



Coding tracker trend chart

6.5. Resume Upload

Resumes upload directly from the browser to Amazon S3 using a presigned URL, confirming that the file-storage path works without routing file bytes through Lambda or API Gateway.



The screenshot shows the 'Smart Placement' application's 'Profile' page. The left sidebar contains navigation links: Dashboard, Profile (selected), Companies, Aptitude, Coding, and Analytics. The main content area is divided into several sections: a top section for personal details (Phone, College, Branch, CGPA, LinkedIn URL, GitHub URL, LeetCode profile, HackerRank profile), a 'Certifications' section with an 'Add certification' button, a 'Projects' section with an 'Add project' button, and a 'Resume' section. The 'Resume' section features a file upload button labeled 'Choose File' and a status message 'No resume uploaded yet.'

Resume upload on the profile page

6.6. Analytics

The analytics page correctly renders applications-per-month, selection-rate, coding-growth, and aptitude-improvement charts from a student's live data.



Analytics page with aggregated charts

6.7. Scalable, Serverless Architecture

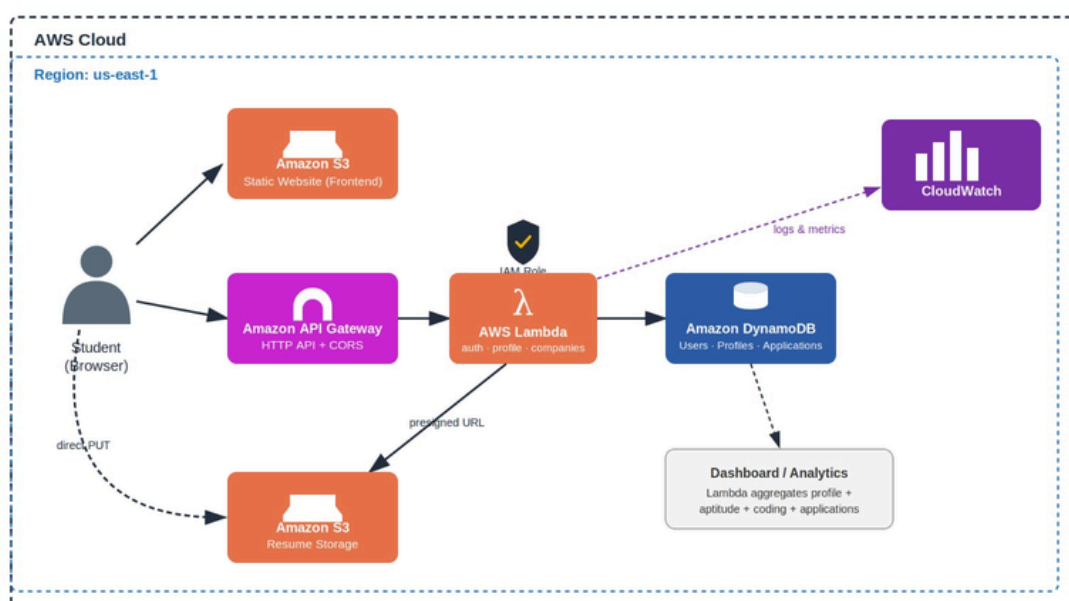
The system handles requests without any manual scaling or downtime, thanks to the underlying serverless AWS infrastructure. The combination of S3, API Gateway, Lambda, and DynamoDB ensures that each component functions independently and scales automatically based on demand, at near-zero cost while idle.

6.8. Secure and Controlled Service Access

Using AWS IAM roles and policies, secure access was configured between all services so that each Lambda function can only reach the specific DynamoDB table or S3 bucket it needs, validating that the system follows least-privilege best practice.

The overall results demonstrate that Smart Placement Tracker meets its intended goals of automation, efficiency, scalability, and ease of use. The architecture is stable and ready for real-world use by placement-prep students, with clear opportunities for further expansion.

Architecture:



7. CONCLUSION

The successful development and deployment of Smart Placement Tracker demonstrates the practicality of cloud-native, event-driven architectures in solving a real, everyday student problem. By leveraging key AWS services – S3, API Gateway, Lambda, DynamoDB, and IAM – the system achieves its core objectives: giving a student one place to manage their profile, track applications, log practice, and see an honest, automatically calculated view of their placement readiness, all without managing any physical infrastructure.

The project highlights the benefits of serverless computing, including automatic scaling, cost-efficiency, and reduced operational complexity. Lambda functions allow on-demand processing of every request, DynamoDB provides fast and reliable storage keyed for cheap per-student queries, and S3 handles both static hosting and direct file uploads without adding load to the compute layer.

Throughout the development lifecycle, attention was given to security, performance, and modularity: IAM permissions were scoped to least privilege, CloudWatch logging was used throughout for debugging integration issues, and a shared Lambda layer kept common logic from being duplicated across functions. Testing confirmed the system's ability to handle the full student workflow reliably, end to end.

Beyond its current functionality, the project lays a strong foundation for future enhancement, such as:

- An AI Career Assistant chatbot (via Amazon Bedrock or a similar LLM API) for interview prep, resume feedback, and roadmap guidance
- Automated deadline and low-readiness notifications via Amazon SNS
- An admin dashboard for placement-cell staff to view aggregated, anonymized cohort statistics
- A readiness-history table to power a genuine week-over-week trend chart
- AI-assisted resume ATS scoring immediately after upload

In conclusion, this project serves as a practical, scalable blueprint for serverless application development in the education-technology space, showing how cloud services can be combined to build an efficient, secure, and future-ready placement preparation tool with minimal operational burden.